TSE - Assignment
Metamorphic Testing of a Multi-Agent Housing-Trends Application

Due· October 16, 2026; Teams of 2

1. What you will build

Build a small multi-agent application that analyzes residential housing listings in the Dallas, Texas metropolitan area and reports how prices have moved over the past twelve months. Several cooperating agents each scan listings against a set of user-specified criteria, and the system aggregates their findings into a one-year trend comparison. There must be a nice visual interface that users can interact with.

The agentic workflow must **not be a linear pipeline**. Your design should include at least: (a) one **feedback loop** where a downstream agent sends work back upstream, (b) a **conditional/iterative control step** where an agent decides whether to stop or iterate based on a signal such as confidence or coverage, and (c) a genuine **dependency** where one agent's output changes what another agent does next, not merely what data it receives. Your design note (to be finalized and submitted in first 10 days) must show this control flow as a graph (with the cycles marked) and justify why the loops are necessary rather than decorative. In your final report, also include a short execution trace/log showing a loop actually firing at run time.

Data source. Use any accessible source and document it: a public/edu real-estate dataset, a synthetic feed that you created, or a decent sized scraped/exported sample. **You must ship a frozen local snapshot with your code** so results are reproducible. *Remember, live sites change, and your grade depends on re-runnable numbers !*

2. Metamorphic testing

Rather than checking each output against a known-correct value, you will define **metamorphic relations (MRs)**. You transform an input, re-run the system, and check that the two outputs relate as the property demands. A violated relation exposes a fault without any oracle for the individual answer.

Required: *Design and empirically evaluate at least eight MRs* spanning all three families (below). For each MR, state the relation, the input subdomain on which it should hold, the tolerance, and the fault it would expose if violated. The eight relations in the table are illustrative. You may adopt, modify, or replace them, but keep all three families. Before writing any MR, define your output contract: the exact set of statistics that make up "the 12-month trend" (e.g., per-month median $/sqft and active-listing count). Every MR is a statement about these specific outputs.

Example metamorphic relations (you may extend/modify these)

Family	Transformation	Expected relation (the MR)
Invariance	Reorder / relabel the input listings; re-run	The 12-month trend and all summary statistics are unchanged, order must not matter.
Invariance	Change currency display or units (USD↔USD-in-thousands; sqft↔same)	Reported percentages and rankings are identical; only display scales.
Invariance	Duplicate every listing (2× the dataset)	Medians and $/sqft trends are unchanged; only the active count doubles.
Monotonic	Add $X to every list price in one month	That month's median list price rises by ~$X; no month decreases.
Monotonic	Widen the price-band criterion (superset of listings)	The active-count series is ≥ the original series, month by month.
Monotonic	Restrict to a sub-region (subset of ZIPs)	Each monthly count ≤ the all-region count for the same month.
Directional	Shift the 12-month window forward by one month	The 11 overlapping months' statistics match the previous run's overlap.
Directional	Scale all prices by a factor k > 1	Median $/sqft scales by ~k; the shape of the trend (% changes) is invariant.

Statistical checking. Because agents may be non-deterministic, check each MR *statistically*: run each relation over at least 30 distinct base inputs (criteria sets / slices), report the violation rate with a 95% confidence interval, and fix and record all seeds. The ≥30 minimum refers to distinct base inputs per MR, not total runs. For non-deterministic systems, repeat each

base input under $k \geq 3$ fixed, recorded seeds (source and follow-up), and report the seed list, base-input count, violation rate, and 95% Confidence Interval (CI). When the CI is tight enough to support your conclusion you can conclude that you have "enough" tests.

Diagnosing violations: For your three most-violated MRs, determine the cause with evidence: a genuine agent/aggregation fault, a data artifact (bad listings), an ill-conceived MR (the property does not actually hold, e.g., duplicates legitimately change a "unique sellers" metric), or a harness bug. Place emphasis on the diagnosis.

3. Deliverables

- Design note: agent architecture + data source + 8 MR sketches
- Working application on the frozen snapshot + MR catalog with justifications
- Metamorphic harness executed; violation rates with Cis
- Reflection (½ page): clearly mention in your report which aspects an LLM could have produced without running your system, and why it could not have produced the rest.

The final report (10-12 pages) must contain details of the system and data source; the MR catalog with justifications; results (violation rates + CIs); root-cause analysis of the top three violations; and a threats-to-validity section. Submit a git repo with code, the frozen data snapshot, NOTEBOOK.md, AI_USE.md, and a one-command reproduction of your headline results table.

4. Ground rules

- You are encouraged to use LLMs/agents to build the application. Disclose all substantive use in AI_USE.md, including cases where the tool was wrong and how you caught it.
- The frozen data snapshot and your seeds make results reproducible; the instructor re-runs a sample. Live-only results that cannot be reproduced score zero on rigor.
- An MR is a hypothesis, not an axiom. So, if it does not actually hold for your metric, state it clearly and refine it. Reasoning precisely gets you the grade.
- Use only public, exported, or synthetic data. Do not enter personal data or paid credentials into a listing site.

Open agentic frameworks you may use

All of these are open-source and free to run against a local model (e.g., via **Ollama**) to avoid API costs. Do not shop around for weeks and waste your time. Just pick one as quickly as possible.

Framework	Language	Good for / hint
LangGraph (LangChain)	Python/JS	Graph of agents with explicit state
CrewAI	Python	Role-based "crew" of agents; the fastest way to spin up cooperating roles.
Microsoft AutoGen / AG2	Python	Conversational multi-agent orchestration; strong for agents that talk to each other.
Pydantic AI	Python	Typed, testable agents with structured outputs. This is expected to work well with metamorphic checks.
LlamaIndex / Haystack	Python	Data- and retrieval-oriented agent toolkits if your listings need indexing/RAG.
No framework	Any	Plain LLM function-calling + Python functions + a loop. Perfectly acceptable.

Datasets to get started (with links)

Download once, filter to the Dallas–Fort Worth metro (or Dallas ZIPs, 75xxx), and freeze a local snapshot. Two flavours: **per-listing data** (your agents "scan listings" and you compute your own 12-month trend) and **ready-made monthly indices** (a published trend to cross-check against).

Dataset	Link	What it gives you
Kaggle — USA Real Estate (2.2M listings)	kaggle.com/datasets/ahmedshahriarsakib/usa-real-estate-dataset	Per-listing: price, bed, bath, city, state, zip, house_size, prev_sold_date. Filter to TX/Dallas
Zillow Research (ZHVI / ZORI)	zillow.com/research/data	Monthly home-value & rent indices by metro and ZIP, incl. Dallas. A ready 12-month trend, CSV.
Redfin Data Center	redfin.com/news/data-center/downloads/	Weekly/monthly median sale price, inventory, price drops by metro (Dallas), downloadable TSV.
Realtor.com research / FRED	fred.stlouisfed.org/tags/series?t=realtor.com	Monthly active-listing count & median list price for the Dallas–Fort Worth–Arlington MSA.
FHFA House Price Index (MSA)	fhfa.gov/data/hpi	Quarterly HPI for the Dallas MSA

If you are not able to extract and create dataset from the above links, you may generate a synthetic Dallas listings snapshot (price, beds, sqft, ZIP, month) with a small Python script. This gives you full control for the MRs and stays fully reproducible. Whichever you choose, make sure you cite it in your report.

Getting-started hints

- Keep agents small and deterministic where you can (temperature 0, fixed seeds). Non-determinism is exactly what your statistical MR checking is designed to handle.
- Freeze the data first. Download once into a local CSV/JSON snapshot, then point every agent at the snapshot. This makes your MRs (reorder, duplicate, scale) trivial to apply and your results reproducible.
- Write one MR end-to-end before building all eight as evidence of your harness (transform → re-run → compare with tolerance) on the simplest invariance (reorder listings) first.
- Use a local model (Ollama + Llama/Qwen/Mistral) or free API tier for development. A small model is fine since you are testing the pipeline's behaviour, not chasing benchmark quality.

Optionally you may swap Dallas for a major Indian metro !!!

If you would rather work with a market around you, retarget the entire assignment to a major Indian metro: Bengaluru, Mumbai, Delhi-NCR, Hyderabad, Chennai, or Pune. Nothing else changes: the same MR families and the same statistical protocol. Adapt the criteria to local conventions. For example, you may filter by area/locality (e.g., Koramangala, Powai, Gachibowli) and PIN code rather than ZIP, express size as carpet/built-up area in sq. ft., prices in ₹ lakh/crore, and keep bedroom counts in the Indian "BHK" idiom (1/2/3 BHK). Your currency- and unit-invariance MRs become especially instructive here: converting ₹ crore ↔ ₹ lakh, or reordering listings across localities, must not move the trend or the rankings.

Indian datasets to get started (with links). For per-listing data your agents can scan, use the Kaggle India collections:

House Price Prediction (Delhi, MagicBricks) at kaggle.com/datasets/jatin31/house-price-prediction-delhi-magicbricks, *Bengaluru House Price Data* at kaggle.com/datasets/amitabhajoy/bengaluru-house-price-data, or the multi-city *99acres.com* scrape at kaggle.com/datasets/arnabk123/real-estate-data-sourced-from-99acres-com (Mumbai, Hyderabad, Kolkata, and more).

For a published trend to cross-check against, use the **NHB RESIDEX** official housing price index, which tracks residential prices quarterly across 50 cities (nhb.org.in/about-residex). One caveat to plan around: several of these snapshots are single-point listings rather than clean monthly time series, so, exactly as in the Dallas version, either bucket listings by their sale/scrape date to build your own trend, or use RESIDEX for the year-over-year movement. Whichever city and source you pick, download once into a frozen local snapshot, respect each source's terms of use, and cite it in your report.

NOTE: *This assignment is an experiment in itself and some parts may be incomplete. Also, there may be some ambiguities. Make reasonable assumptions and document the same.*